Pricing options and computing implied volatilities using neural networks

Aleksandra Górska,
Jakub Kempa,
Marcin Pałyga

February 2026

1 Introduction

Traditional option pricing methods such as Monte Carlo simulations or Finite Difference Method are commonly used to quantify an option's theoretical value. Since those methods are based on mathematical theory they often offer high accuracy, however they are computationally slow. Because of this problem usage of those methods may become impossible for tasks such as real-time trading or risk management. To overcome this difficulty Liu et al. [4] propose utilizing Neural Networks for computing options' prices and implied volatilities.

Since then Machine Learning methods became regular framework for pricing options. Once trained they can compute prices instantaneously with accuracy. Also many improvements have been made since the paper came out. We propose using Physics-Informed Neural Networks(PINNs) which embed underlying financial partial differential equations directly into the loss function. This ensures the model respects financial dynamics rather than relying purely on data. Since the model is constrained by financial laws it should not produce any arbitrage opportunities and acts as a golden mean between theoretical methods and machine learning methods.

The aim of this project is to generate synthetic data using Black-Scholes PDE, train Artificial Neural Network and Physics-Informed Neural Networks on said data and compare the obtained results. In the last section there will also be evaluation of results in comparison with the 2019 paper *"Pricing Options and Computing Implied Volatilities using Neural Networks"* [4].

Table 1: Comparison of methods

Methodology	**Main Advantage**	**Main Weakness**
Monte Carlo / Finite Difference	Theoretically Exact	Too slow for real-time
Artificial Neural Network	Instantaneous	Allows arbitrage
Physics-Informed Neural Network	Theory-consistent & Fast	Requires longer training

2 Framework

The outline of this project can be separated into the following steps:

1. Preparing input parameter ranges.
2. Generating data for training Artificial Neural Network (ANN).
3. Hyperparameter search for the ANN's optimization.
4. Splitting data into training, validation and test sets.
5. Training the ANN.
6. Evaluating the results.
7. Implementation of Physics Informed Neural Network (PINN).
8. Training the PINN.
9. Comparing results of both models' fit.
10. Calculating implied volatility and evaluating the results.

The implementation of both networks will be performed using *Python* with the use of necessary libraries:

- Pandas,
- NumPy,
- PyTorch.

The library used for visual representation of results is Matplotlib.

The main idea was to implement, train and compare the two Neural Networks: Artificial Neural Network and Physics Informed Neural Network. The data that these models will have been trained is synthetic data, generated using Black-Scholes closed-form European option price equations.

Our aim is to reflect the methodology and results of the 2019 paper *"Pricing Options and Computing Implied Volatilities using Neural Networks"* [4] for option pricing Neural Network.

2.1 Option pricing with the Black-Scholes PDE

The asset model utilized in this project was the geometric Brownian Motion (GBM) asset model, which is used in the Black-Scholes option pricing PDE.

The GBM asset prices model is represented as such:

$$dS_t = \mu S_t dt + \sqrt{v} S_t dW_t^s, \tag{1}$$

where:

- S - the price of a non-dividend paying asset,
- W^s - Wiener process,
- t - current time,

- μ - drift parameter,
- v - variance parameter.
- $\sigma = \sqrt{v}$ - volatility parameter.

Deriving the Black-Scholes PDE from Itô's Lemma [5] we obtain:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rs\frac{\partial V}{\partial S} - rV = 0, \tag{2}$$

with $V(t, S)$ being the option price and r the risk-free interest rate.

Our focus was set on the European option contracts, which adds final condition, representing the payoff:

$$V(T, S) = (S_0 - K)^+, \tag{3}$$

where K is the option's strike price.

A closed-form solution for these set of equations is represented by:

$$V(t, S) = S\Phi(d_+) - Ke^{-r\tau}\Phi(d_-), \tag{4}$$

$$d_+ = \frac{\log(S/K) + (r - \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \quad d_- = d_+ - \sigma\sqrt{\tau}, \tag{5}$$

where $\tau = T - t$ (time to maturity T) $V(t, S)$ is the European call option value at time t for stock value S and $\Phi(x)$ is the cumulative distribution function of the normal distribution for given value x.

This equation will be used for synthetic data generation needed for Neural Network training.

2.2 Data Generation

For the synthetic data we generated 100 000 random samples for the input parameters S_0, τ, r, σ. The range of input parameters are presented in table 2.

Table 2: Black-Scholes parameter ranges used in synthetic data generation

Parameter	**Range**	**Unit**
initial asset value (S_0)	$0.4 - 1.6$	-
time to maturity (τ)	$0.2 - 1.1$	years
risk-free rate (r)	$0.02 - 0.1$	-
volatility (σ)	$0.01 - 1.0$	-

We have set $K = 1$ without loss of generality because European option prices are homogeneous of degree one in (S, K), so dividing all prices and inputs by K reduces dimensionality while preserving the model's structure and accuracy.

Samples were generated utilizing Latin hypercube sampling [6].

This approach resulted in 100 000 random samples, each containing five variables $\{S_0, \tau, r, \sigma, V\}$. The training set includes $\{S_0, \tau, r, \sigma\}$ as input, while the output is the option price V.

3 Neural Networks

3.1 Neural Network architecture

The most important part of the report - the Artificial Neural Network (ANN) - was implemented. The ANN constitutes three parts, i.e. neurons, layers and the overall architecture. At the bottom of the structure is a neuron - fundamental unit of ANNs, that involves learnable weights and biases. Each layer has a fixed number of neurons, which are connected with neurons from adjacent layers. This connection provides the output signal of a neuron from previous layer as the input signal for the neuron in the next layer. The architecture with multiple hidden layers makes the input layer pass to the output layer, allowing the ANN to build a mapping among input-output pairs.

The mathematical formula for the function that the ANN defines is

$$y(\mathbf{x}) = F(\mathbf{x}|\theta), \tag{6}$$

Where $\mathbf{x}$ is the vector of input parameters (features) and the θ is a vector of weights and biases

$$\theta = (\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, \cdots, \mathbf{W}^{(l)}, \mathbf{b}^{(l)}). \tag{7}$$

For the L neural layers $\mathbf{W}^{(l)}$ is a weight matrix and $\mathbf{b}^{(l)}$ is a bias vector of the l-th layer.

Denoting the value of the j-th neuron we obtain

$$z_j^{(l)} = \varphi^{(l)} \left(\sum_i w_{ij}^{(l)} z_i^{(l)} + b_j^{(l)}, \right) \tag{8}$$

where $\varphi(x)$ is an activation function.

The activation function adds non-linearity to the system. The activation functions employed in the project are:

- ReLU, $\varphi(x) = \max(x, 0)$,
- ELU, $\varphi(x) = x \cdot 1_{x \geq 0} + \alpha(e^x - 1) \cdot 1_x < 0$,
- tanh, $\varphi(x) = \tanh(x)$ (used for PINNs in the later section).

The objective of the ANN is to minimize the loss function defined for ANNs as

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} ||\mathbf{y} - \hat{\mathbf{y}}||_2, \tag{9}$$

where $\mathbf{y}$ is the vector of true values in the dataset, while $\hat{\mathbf{y}}$ is the vector of predicted values, based on provided vector of features $\mathbf{x}$. This metric is exclusively used for the ANN implemented in the project - the PINN's loss function adds weighing of different MSE functions, which will be later explained in section 4.

For robust NN model, several back-propagation gradient descent methods were tested. These optimization algorithms

This neural network architecture is appropriate for the presented problem, due to the universal approximation theorem, which states that neural networks can, in principle, approximate any continuous function to any desired degree of accuracy [3], which fits the use of Black-Scholes equation 2.

3.2 Hyperparameter search

For the NN to achieve the best results, hyperparameters of the structure have to be chosen correctly. The hyperparameters include the number of layers, neurons, activation function, etc. Found set of values will later be used in training the model.

Hyperparameter tuning was performed using random search over a pre-defined grid of values for each hyperparameter. The choice of random search over the grid search was done due to higher efficiency [1][4].

The setting of random search for hyperparameters optimization can be shown in table 3. The dataset chosen for random search was 10% of the original set, resulting in 10 000 randomly selected samples. This helps reduce the computational requirements, while still keeping the results valuable.

For each combination of parameters, cross-validation using 3 splits was performed and each cross-validation combination was tested over 50 epochs. The best set chosen was the set with the smallest loss value - later used for training. The results of the search can be seen in table 4.

The results favor the ReLU activation function and the Adam optimizer, which is consistent with their known robustness and efficiency in training NNs. Batch normalization was selected, suggesting improved training stability for this architecture.

Table 3: Hyperparameter grid definition for NN random search

Parameters	**Options / range**
Activation function	relu, elu
Hidden neurons	$n \in [200, 600]$
Optimizer	SGD, RMSprop, Adam
Learning rate	$\mathrm{lr} \in [10^{-4}, 10^{-2}]$
Use batch normalization	True, False
Batch size	$b \in [128, 2048]$

Table 4: Set of hyperparameters with the smallest loss

Name	**Value**
Activation function	relu
Hidden neurons	587
Optimizer	Adam
Learning rate	0.00653
Use batch normalization	True
Batch size	223

3.3 Neural Network full implementation

After the results from hyperparameter search were obtained, the training setup and process was performed. The full dataset was split into three parts:

- training set - 80% of the samples,
- validation set - 10% of the samples,
- test set - 10% of the samples.

The training set was used for training the model. The validation set was used to monitor generalization performance during training. This prevented overfitting by selecting the model state that minimizes validation loss, rather than training loss. Final evaluations were performed with the use of the test set, comparing the predictions based on the test set features, to the test set labels.

The training was done for 3000 epochs and the results of the fit, further seen in section **??** in comparison with the PINN model fit results, are more than satisfactory.

With the R^2 defined as

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{10}$$

where y_i are the actual values, $\hat{y}_i$ the predicted values, $\bar{y}$ the mean of actual values, and n is the number of samples.

Figure 1 shows that the R^2-score (seen on the left subplot) represents a near-perfect fit, meaning that the model explains $\approx 99.9933\%$ of the variance in the target variable. The right subplot shows the residuals plot. Presented values are very small and lie roughly centered around zero. These results show high accuracy and validate the methodology used in training the model.

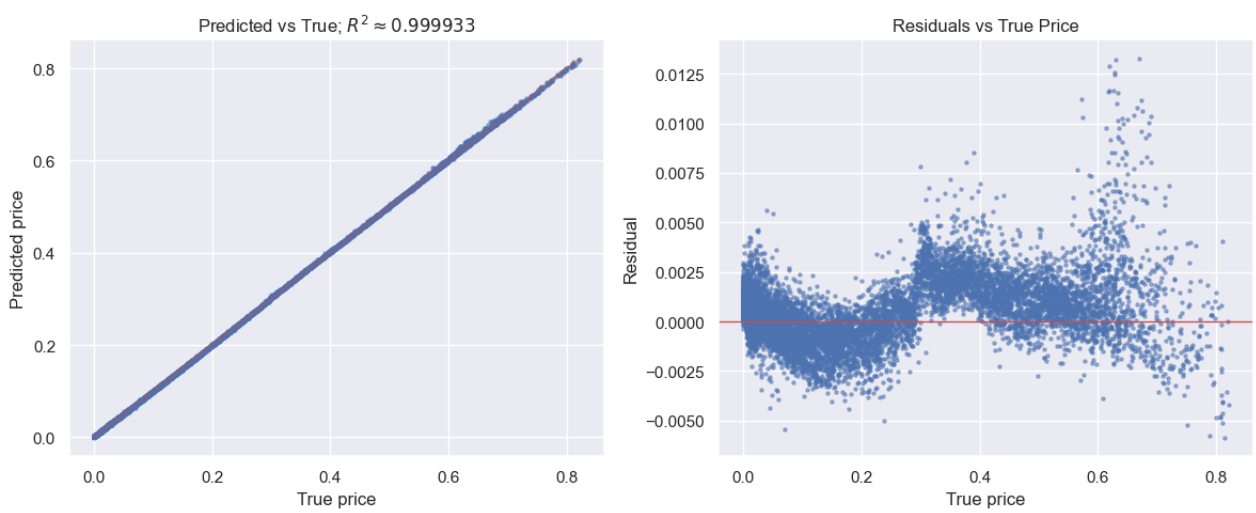

Figure 1: R^2 score results and residuals plot for fitted NN model

4 Physics-Informed Neural Networks

As mentioned earlier Artificial Neural Networks suffer from brute-force pattern recognition relying purely on data without understanding any of the economic laws governing the market. As a result they may predict prices that violate the absence of arbitrage. Physics-Informed Neural Networks have been introduced as a way to address those limitations. The core of PINNs lies in their loss function. Unlike ANNs that minimize only the Mean Squared Error, PINNs incorporate a physics loss term. It works as a constraint that penalizes the model when its predictions fail to satisfy the underlying PDE which in our case was the Black-Scholes PDE. The network can compute exact derivatives required by the equation and then learn the entire dynamical structure of the option's value.

Another important advantage of PINNs is their ability to address the "curse of dimensionality". Traditional numerical methods like FDM rely on grid-based discretization of the domain, so as the number of underlying assets increases, the number of grid points grows exponentially. Since PINNs do not use a discrete grid they are capable of pricing high-dimensional derivatives that are computationally difficult for FDM or Monte Carlo methods. PINNs can also be used for risk management. The Greeks are critical for assessing risk as they quantify sensitivity of derivatives to price changes, volatility or interest rates. In a PINN framework the Greeks are obtained during learning of the model and then used to calculate the PDE-based part of the loss function.

For our implementation of Physics-Informed Neural Network we did not use random search for hyperparameters as we did for ANN. The Tanh activation function is the standard choice for PINNs as it is differentiable which is required for our loss function. Because derivatives must be calculated at every training step, training PINNs is more computationally expensive than training standard ANNs. For this reason we set the hyperparameters values to common values frequently used as a starting point for training NNs. Hyperparameters for PINN are presented in Table 5.

Table 5

Parameters	**Options / range**
Activation function	Tanh
Hidden neurons	64
Optimizer	Adam
Learning rate	0.001
Use batch normalization	False
Batch size	None

The loss function that we wanted to minimize using PINN can be defined as

$$L_{total} = L_{data} + \lambda L_{pde}, \tag{11}$$

where L_{data} ensures the model matches the market prices (same as loss function for ANN), L_{pde} penalizes the model if it do not satisfy the Black-Scholes PDE and λ is a hyperparameter that balances the importance of theory in our loss function [2].

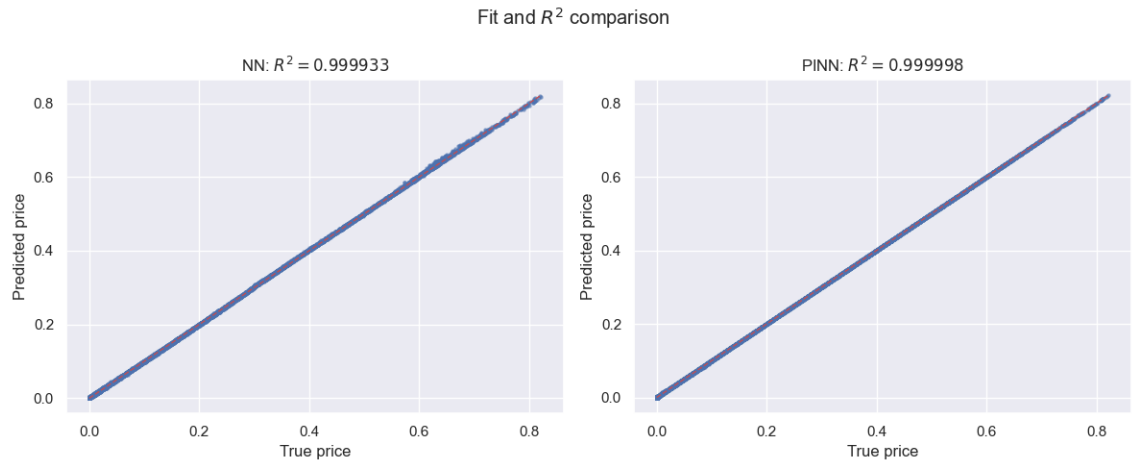

Figure 2: Comparison of fit and R^2 values on test set

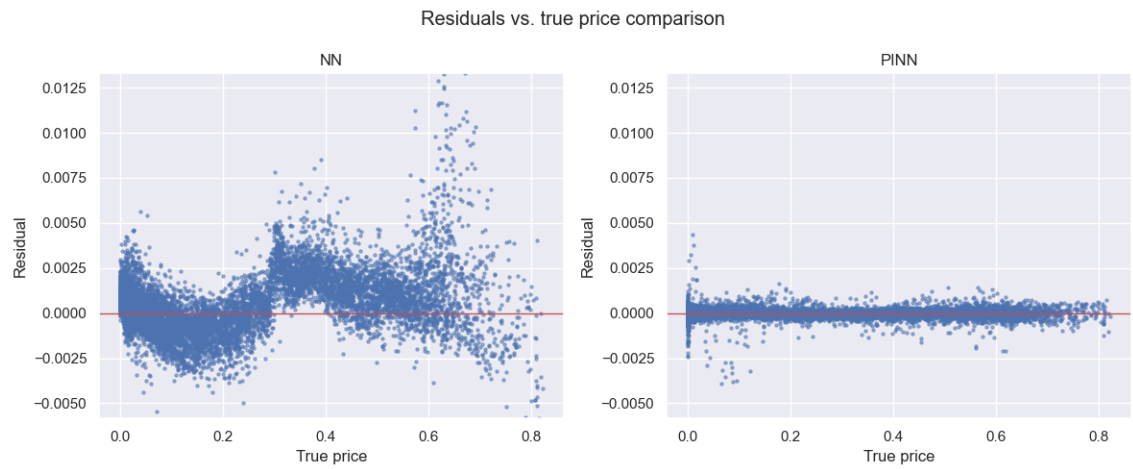

Figure 3: Comparison residuals to true price on test set

5 Comparison of results

In this chapter, the results of proposed methods are compared to evaluate their performance and to identify key differences in accuracy and behavior. Figure 2 illustrates performance of the Artificial Neural Network and the Physics-Informed Neural Network on the test set. Both models exhibit a nearly perfect fit with PINN achieving slightly higher R^2 value than ANN. To further investigate the reliability of the models, Figure 3 presents a comparison of residuals against true option prices. The ANN shows higher dispersion and some waving pattern, suggesting it fails to capture dynamics of option prices. In contrast, PINN residuals are uniformly distributed and more tightly clustered around the zero axis. Figure 4 provides a summary of error profiles for both models using density

comparison of residuals. PINN produces highly concentrated distribution centered around zero. However, ANN exhibits a broad distribution with higher variance and fat tails. Figure 5 indicates that ANN is prone to unpredictable outliers and that PINN demonstrates a much tighter alignment with the theoretical quantiles. These plots consistently confirm the superiority of PINN for this task. While standard Neural Networks can approximate prices, they often do so with bias and unpredictable outliers. By integrating the Black-Scholes PDE, PINN yields prices that are not only more accurate but also theoretically consistent.

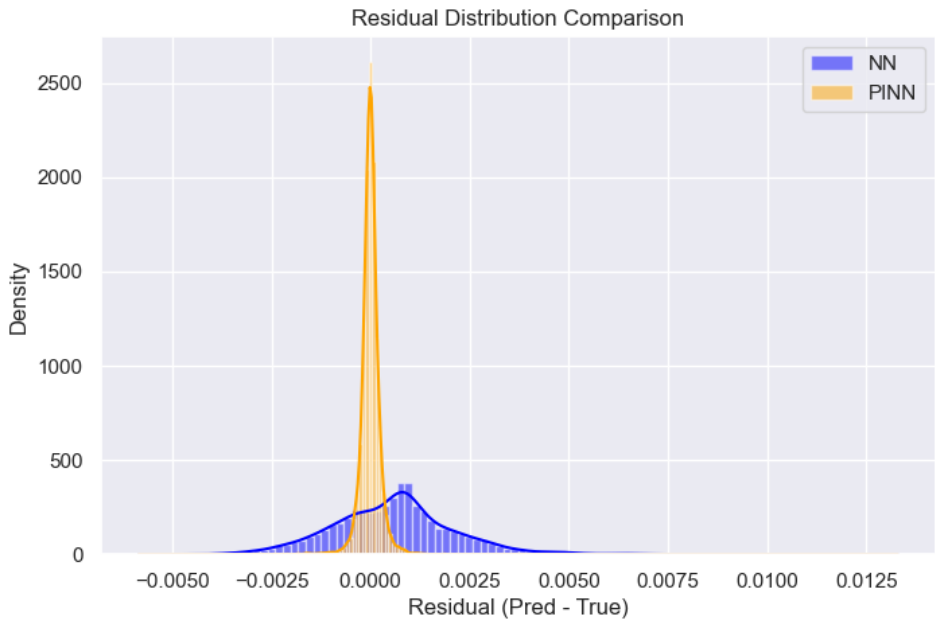

Figure 4: Residual distribution comparison

Implied volatility is considered an important quantity in finance. It represents market's forecast of likely movement in security's price. As shown in [4] we can write the implied volatility as an implicit formula,

$$\sigma^*(K,T) = BS^{-1}(V; S, K, \tau, r), \tag{12}$$

where BS^{-1} denotes the inverse Black-Scholes function. An analytic solution to this equation does not exist and it is determined by means of a numerical iterative technique, as it can be converted into a root-finding problem. We use Brent's method to compute implied volatility surfaces which can be seen on Figures 6 and 7. The Artificial Neural Network produces an erratic surface with numerical noise, which could lead to potential arbitrage violations in live trading. In contrast the PINN produces a smooth, continuous surface with small differences in values, as the surface should be flat when using data generated by Black-Scholes model. An even bigger difference can be seen for wider maturity range where the surface produced by ANN collapses for $\tau > 1.5$ and PINN still produces smooth continuous surface, however with higher differences between minimum and maximum values. By computing implied volatility surfaces we can identify another important advantage of PINNs over Artificial Neural Networks in financial applications.

Figure 5: QQ-plot comparison

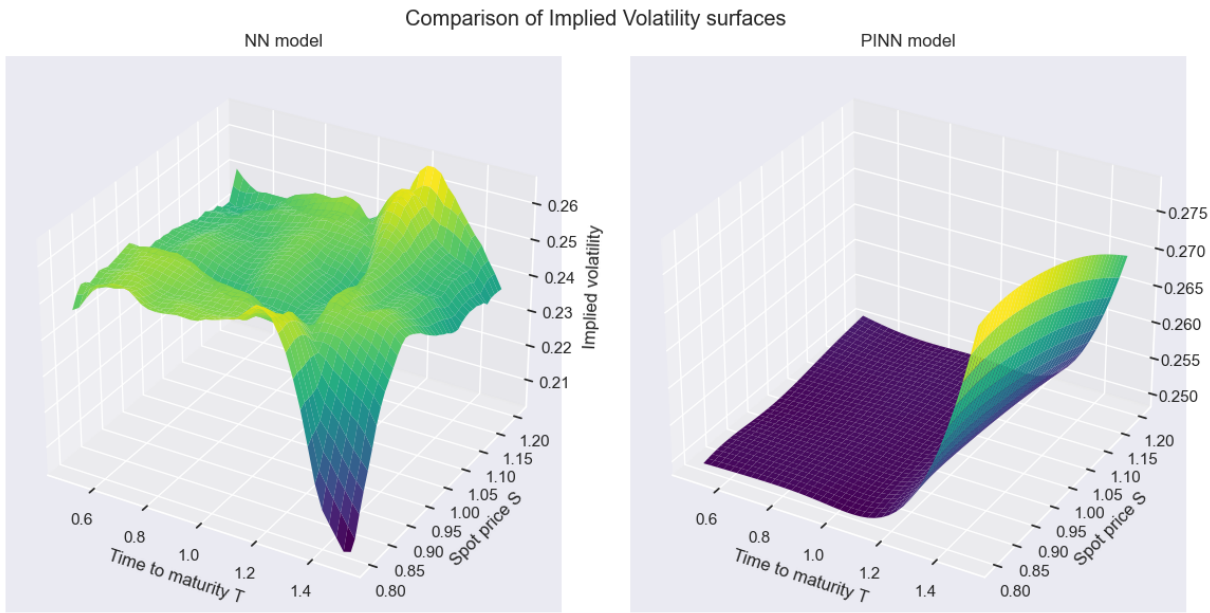

Figure 6: Implied volatility surface

6 Conclusions

The Neural Networks approach clearly shows accurate results in predicting the European call option prices. Random search was found to be an effective method for optimal hyperparameter search, resulting in high R^2 scores for the ANN predictions.

The implementation of PINN is justified, given the PDE, that describes the data generation.

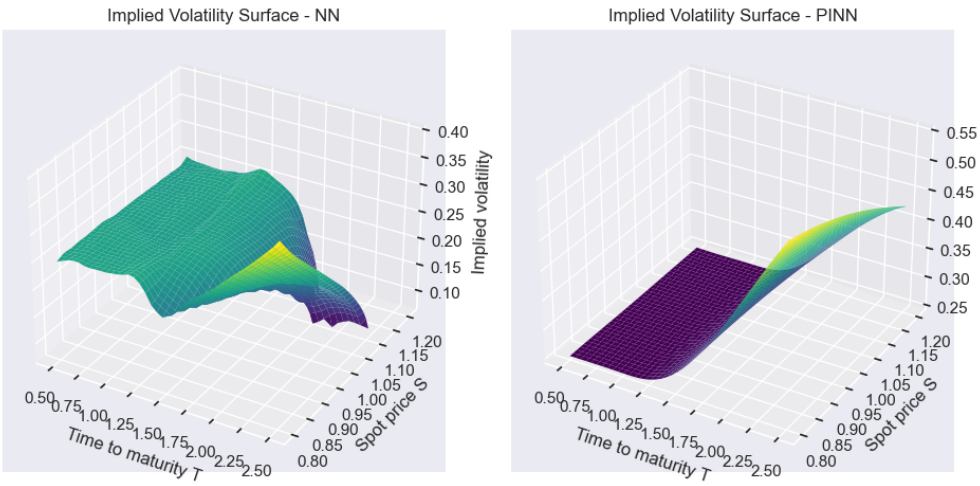

Figure 7: Implied volatility surfance - Long maturity

The result obtained from fitting the PINN to the synthetic data shows improvement, over the ANN fit results, which strengthens the reasoning in utilizing the PINN for clearly-defined PDE problems.

The obtained implied volatility surfaces show good predictions of prices for maturity times $\tau \leq 1.2$, but suggest more training and better dataset for higher maturities, due to instability and blow-outs of the implied volatilities.

Part of the challenge of this project was the computational requirements, that prevented performing a random search for PINN architecture. Nonetheless, the choice of common PINN hyperparameter values resulted in a very good fit and predictions, which can leave one optimistic for the results obtained after the hyperparameter search.

Unfortunately our implementation of the ANN does not return the same results as the Liu et al. article [4]. The major contributing factor to this was the computational deficiency. Nevertheless the obtained results are more than satisfactory.

References

[1] Bergstra, J., Bengio, Y. Random search for hyper-parameter optimization. *The journal of machine learning research 13*, 1 (2012), 281–305.

[2] Dhiman, A., Hu, Y. Physics informed neural network for option pricing. *arXiv preprint arXiv:2312.06711* (2023).

[3] Hornik, K., Stinchcombe, M., White, H. Multilayer feedforward networks are universal approximators. *Neural networks 2*, 5 (1989), 359–366.

[4] Liu, S., Oosterlee, C. W., Bohte, S. M. Pricing options and computing implied volatilities using neural networks. *Risks 7*, 1 (2019), 16.

[5] Malliaris, A. G. *Stochastic Methods in Economics and Finance*. North-Holland, New York, 1982.

[6] McKay, M. D., Beckman, R. J., Conover, W. J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics 42*, 1 (2000), 55–61.